andreasveit / densenet-pytorch Public

A PyTorch Implementation for Densely Connected Convolutional Networks (DenseNets)

BSD-3-Clause license

409 stars 133 forks

Star Watch

Code Issues 5 Pull requests Actions Projects Security Insights

master

andreasveit ... on 28 Feb 2018

View code

A PyTorch Implementation for Densely Connected Convolutional Networks (DenseNets)

This repository contains a PyTorch implementation of the paper Densely Connected Convolutional Networks. The code is based on the excellent PyTorch example for training ResNet on Imagenet.

The detault setting for this repo is a DenseNet-BC (with bottleneck layers and channel reduction), 100 layers, a growth rate of 12 and batch size 64.

The Official torch implementaion contains further links to implementations in other frameworks.

Example usage with optional arguments for different hyperparameters (e.g., DenseNet-40-12):

```
$ python train.py --layers 40 --growth 12 --no-bottleneck --reduce 1.0 --name DenseNe
```

DenseNets

DenseNets [1] were introduced in late 2016 after to the discoveries by [2] and [3] that residual networks [4] exhibit extreme parameter redundancy. DenseNets address this shortcoming by reducing the size of the modules and by introducing more connections between layers. In fact, the output of each layer flows directly as input to all subsequent layers of the same feature dimension as illustrated in their Figure 1 (below). This increases the dependency between the layers and thus reduces redundancy.

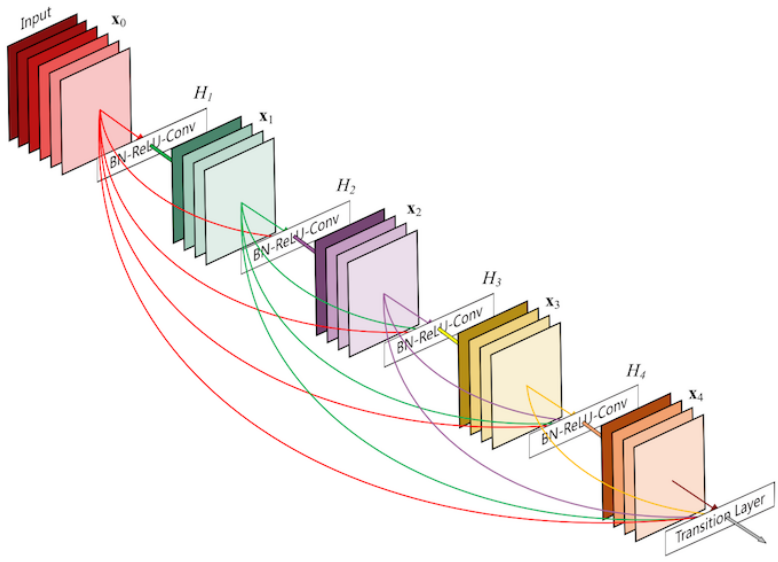
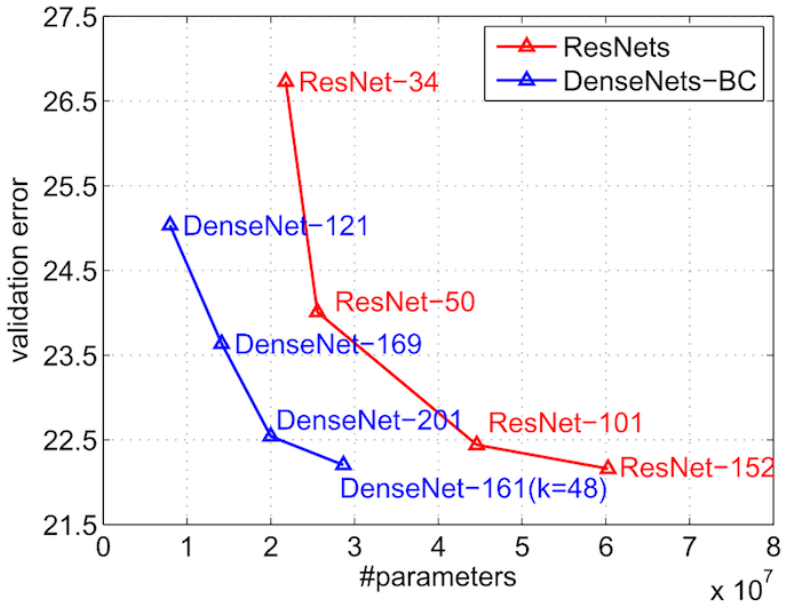


Figure 1. A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

The improvements in accuracy per parameter are illustrated in their results on ImageNet (Figure 3).



This implementation

The training code in train.py trains a DenseNet on CIFAR 10 or 100. To train on ImageNet, densenet.py can be copied into the PyTorch example for training ResNets on Imagenet, upon which this repo is based. Note that for ImageNet the model contains four dense blocks.

This implementation is quite *memory efficient requiring between 10% and 20% less memory* compared to the original torch implementation. We optain a final test error of 4.76 % with DenseNet-BC-100-12 (paper reports 4.51 %) and 5.35 % with DenseNet-40-12 (paper reports 5.24 %).

This implementation allows for **all model variants** in the DenseNet paper, i.e., with and without bottleneck, channel reduction, data augmentation and dropout.

For simple configuration of the model, this repo uses `argparse` so that key hyperparameters can be easily changed.

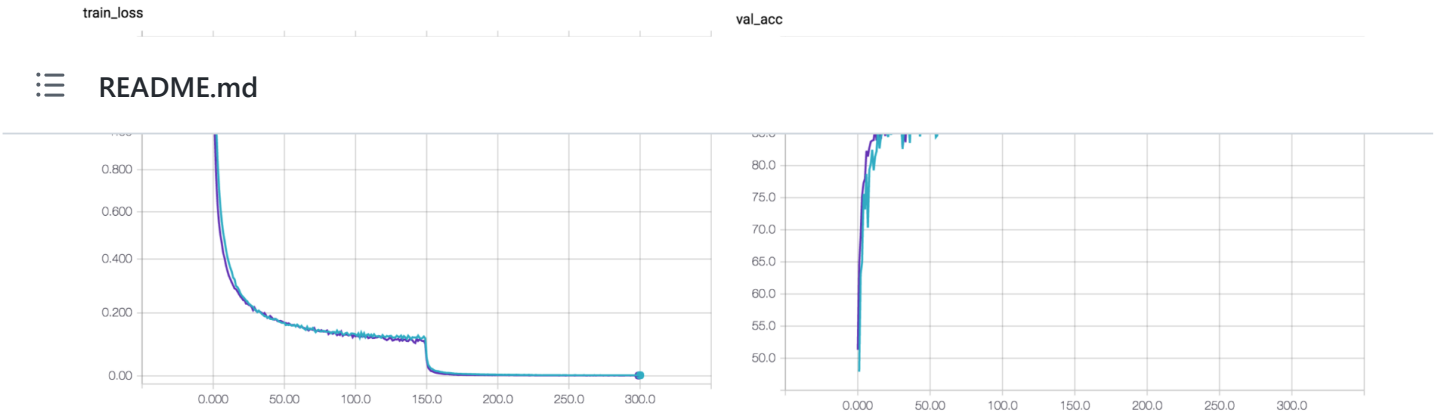
Further, this implementation supports easy checkpointing, keeping track of the best model and resuming training from previous checkpoints.

Tracking training progress with TensorBoard

To track training progress, this implementation uses TensorBoard which offers great ways to track and compare multiple experiments. To track PyTorch experiments in TensorBoard we use tensorboard_logger which can be installed with

```
pip install tensorboard_logger
```

Example training curves for DenseNet-BC-100-12 (dark blue) and DenseNet-40-12 (light blue) for training loss and validation accuracy is shown below.



(a) training loss (b) validation accuracy

Dependencies

- PyTorch

optional:

- tensorboard_logger

Cite

If you use DenseNets in your work, please cite the original paper as:

```
@article{Huang2016Densely,
  author  = {Huang, Gao and Liu, Zhuang and Weinberger, Kilian Q.},
  title   = {Densely Connected Convolutional Networks},
  journal = {arXiv preprint arXiv:1608.06993},
  year    = {2016}
}
```

If this implementation is useful to you and your project, please also consider to cite or acknowledge this code repository.

References

[1] Huang, G., Liu, Z., Weinberger, K. Q., & van der Maaten, L. (2016). Densely connected convolutional networks. arXiv preprint arXiv:1608.06993.

[2] Huang, G., Sun, Y., Liu, Z., Sedra, D., & Weinberger, K. Q. (2016). Deep networks with stochastic depth. In European Conference on Computer Vision (ECCV '16)

[3] Veit, A., Wilber, M. J., & Belongie, S. (2016). Residual networks behave like ensembles of relatively shallow networks. In Advances in Neural Information Processing Systems (NIPS '16)

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Conference on Computer Vision and Pattern Recognition (CVPR '16)

Releases

No releases published

Packages

No packages published

Contributors 3



andreasveit Andreas Veit



mileyan Yan Wang



taineleau-zz Danlu Chen

Languages

- **Python** 100.0%